

# CSCI 235: Software Design & Analysis II

## Day 2 — Inheritance & Polymorphism

Tong Yi

Hunter College, CUNY

Fall 2026

# Outline

- 1 Questions from Last Class
- 2 A Function vs. a Method
- 3 Review: The Is-A Relationship
- 4 Implementing Polymorphism in C++
- 5 The Slicing Pitfall
- 6 Overload vs. Override

## Where we left off

*“Next time: what if one class is a specialized version of another? `MultipleChoice` is a kind of `Question`; `Square` is a kind of `Rectangle`.”*

- Today we make that idea precise: this **is-a** relationship is called **inheritance**.
- Inheritance unlocks **polymorphism** — one function written for the superclass that works with objects of different subclasses.
- Both examples get a subclass today: `Question` gains `ChoiceQuestion`; `Rectangle` gains `Square`.

## Today's goal

Understand `virtual`, method override, and a subtle pitfall called **object slicing** — then apply all of it in today's lecture slip, where you will write `Square.cpp` yourself.

- 1 Questions from Last Class
- 2 A Function vs. a Method
- 3 Review: The Is-A Relationship
- 4 Implementing Polymorphism in C++
- 5 The Slicing Pitfall
- 6 Overload vs. Override

## The Ternary Operator ?:

- `condition ? valueIfTrue : valueIfFalse` is a compact if/else that produces a *value*, not a statement.
- That matters because a member initializer list needs a *value* for each member — you cannot write a full if/else statement there.

from `Rectangle.cpp`

```
Rectangle::Rectangle(double length, double width)
: length_(length > 0 ? length : 2),
  width_(width > 0 ? width : 1) {
}
```

Same idea as: `double L; if (length > 0) { L = length; } else { L = 2; }` — just written as one expression instead of four lines, so it fits inside the initializer list.

## Why a Member Initializer List?

- Members (and base classes) are initialized *directly* in the initializer list, before the constructor body ever runs.
- Without an initializer list, members are initialized before the constructor body runs, and then the body may *assign* new values — extra work, and sometimes impossible (const members, reference members, and base classes without a no-argument constructor all *require* initialization in the initializer list).

from Square.cpp — a mistake worth seeing once

```
// WRONG: creates a temporary Rectangle; does NOT
// initialize the Square's own Rectangle part.
Square::Square(double side) {
    Rectangle(side, side);
}

// Correct: initializes the Rectangle part directly.
Square::Square(double side)
    : Rectangle(side > 0 ? side : 1, side > 0 ? side : 1) {
}
```

# The `this` Keyword and the `::` Symbol

## `this`

- A pointer, available inside every (non-static) member function, to the object the function was called on.
- Its type is `ClassName*`.
- Most common beginner use: `return *this;`, to return the object itself.

```
Rectangle& Rectangle::operator=(  
    const Rectangle& rect) {  
    length_ = rect.length_;  
    width_ = rect.width_;  
    return *this;  
}
```

## `::` (scope resolution)

- Says *which class* a name belongs to.
- Defines a member function outside its class.
- Calls a specific version of a method explicitly.

```
void Rectangle::print() const {  
    ...  
}  
  
// inside ChoiceQuestion::print()  
Question::print(); // explicitly  
the  
// base version
```

## When Do We Use operator=?

- operator= runs when you assign to an object that **already exists**: `rect2 = rect1;` (both objects exist *before* this line).
- That's different from `Rectangle rect2 = rect1;` at the moment of *declaration* — that calls the **copy constructor** instead, because `rect2` doesn't exist yet.
- If you don't write your own, the compiler generates one automatically that copies each member — which is exactly what ours does by hand below. You'd need to write your own once a class manages a resource (like dynamically allocated memory) that a plain member-by-member copy would handle incorrectly — not yet the case for `Rectangle` or `Question`.

from `Rectangle.cpp`

```
Rectangle& Rectangle::operator=(const Rectangle& rect) {  
    length_ = rect.length_;  
    width_ = rect.width_;  
    return *this;           // enables a = b = c;  
}
```

## A Few Logistics Questions

- **Are dropped projects/assignments staggered, or all at once?**  
*[instructor to answer in class]*
- **Is there extra credit?**  
*[instructor to answer in class]*
- **What is the Gradescope course code?**  
*[instructor to answer in class]*

See the syllabus for the current policy, or ask again if it has changed.

- 1 Questions from Last Class
- 2 A Function vs. a Method
- 3 Review: The Is-A Relationship
- 4 Implementing Polymorphism in C++
- 5 The Slicing Pitfall
- 6 Overload vs. Override

# A Function vs. a Method

- A **function** does not belong to a class. A **method** belongs to a class and works with that class's data members.
- You can think of a method as a function that operates on the data members of a particular object.
- In casual conversation — including in these slides — we often use “function” and “method” interchangeably. Context makes it clear which one we mean.

a **function** (belongs to no class)

```
void print(Rectangle* rectPtr) {  
    rectPtr->print();  
}
```

a **method** (belongs to Rectangle)

```
void Rectangle::print() const {  
    std::cout << "rectangle length: "  
        << length_ << ...  
}
```

Notice the free function `print` even *calls* the method `Rectangle::print()` on its parameter — both live in the same program, doing different jobs.

- 1 Questions from Last Class
- 2 A Function vs. a Method
- 3 Review: The Is-A Relationship**
- 4 Implementing Polymorphism in C++
- 5 The Slicing Pitfall
- 6 Overload vs. Override

# Inheritance: The Is-A Relationship

- Inheritance lets us treat an object as an instance of **its own class** OR as an instance of **its superclass**.
- Example: ChoiceQuestion is a subclass of Question. A ChoiceQuestion object like twoPlusThree can be treated as a ChoiceQuestion *or* as a Question.



Every ChoiceQuestion *is a* Question; every Square *is a* Rectangle. Not the other way around — not every Question is a ChoiceQuestion.

## Difference between Question and ChoiceQuestion

|                                            | Question                                | ChoiceQuestion                                          |
|--------------------------------------------|-----------------------------------------|---------------------------------------------------------|
| an object                                  | usaFounded: "When was the USA founded?" | twoPlusThree: "What is 2 + 3 ?" with options 3, 4, 5, 6 |
| data members                               |                                         |                                                         |
| setText behaves the same?                  |                                         |                                                         |
| setAnswer behaves the same?                |                                         |                                                         |
| checkAnswer behaves the same?              |                                         |                                                         |
| print behaves the same?                    |                                         |                                                         |
| How does ChoiceQuestion store its options? |                                         |                                                         |
| How does ChoiceQuestion add an option?     |                                         |                                                         |

Work through this table together — we'll use print to introduce virtual next.

- **Polymorphism** means a function written for the superclass can work correctly with objects of different subclasses — without being rewritten.
- In C++, the parameter of such a function should be **a pointer (or reference) to the superclass**.

### Why this matters

Without polymorphism, you would need a separate `print` function for every new kind of `Question` you ever invent — `ChoiceQuestion`, `NumericQuestion`, `FreeResponseQuestion`, .... With polymorphism, you write `print` *once*, for `Question`, and it keeps working as new subclasses are added.

- 1 Questions from Last Class
- 2 A Function vs. a Method
- 3 Review: The Is-A Relationship
- 4 Implementing Polymorphism in C++**
- 5 The Slicing Pitfall
- 6 Overload vs. Override

# What Changed Since Day 1?

## Question.hpp — Day 1

```
class Question {  
    public:  
        Question();  
        Question(const std::string&  
            text, const std::string&  
            answer);  
        std::string text() const;  
        bool checkAnswer(  
            const std::string& reply)  
            const;  
    private:  
        std::string text_;  
        std::string answer_;  
};
```

## Question.hpp — Day 2 (new lines)

```
void setText(  
    const std::string& text);  
void setAnswer(  
    const std::string& answer);  
  
virtual void print() const;
```

Two ordinary setters, and one new keyword:  
virtual.

Everything else about Question is unchanged. One keyword is what makes runtime polymorphism possible.

## Step 1: Declare virtual in the Superclass Header (1/2)

```
// Question.hpp
class Question {
public:
    Question();
    Question(const std::string& text, const std::string&
            answer);
    std::string text() const;
    bool checkAnswer(const std::string& reply) const;
    // add in v2
    void setText(const std::string& text);
    void setAnswer(const std::string& answer);
```

Everything above is unchanged from Day 1 — these methods behave exactly as before. The new part is on the next slide.

## Step 1: Declare virtual in the Superclass Header (2/2)

```
// Question.hpp (continued)  
  
virtual void print() const;  
// Keyword virtual is needed for runtime polymorphism.  
// With virtual, the version of print() that matches the  
// actual object is called through a Question& or Question*.  
  
private:  
    std::string text_  
    std::string answer_  
};
```

virtual means a derived class may provide its own version of this method — it can be **overridden** in subclasses.

## Step 2: Don't Repeat virtual in the .cpp (1/2)

```
// ChoiceQuestion.cpp
ChoiceQuestion::ChoiceQuestion() : Question() {
}

ChoiceQuestion::ChoiceQuestion(const std::string& text) : Question(
    text, "") {
}

void ChoiceQuestion::addChoice(const std::string& option, bool
    isAnswer) {
    options.push_back(option);
    if (isAnswer) {
        setAnswer(std::to_string(options.size()));
    }
}
```

Ordinary constructors and addChoice — nothing here needs the word `virtual`; it already appeared once, in the header.

## Step 2: Don't Repeat virtual in the .cpp (2/2)

```
// ChoiceQuestion.cpp (continued)  
void ChoiceQuestion::print() const { // override is not  
    allowed outside a class definition  
    Question::print(); // Call superclass print() to print the  
        question text.  
  
    // options.size() returns size_t, an unsigned  
    // integer type.  
    for (size_t i = 0; i < options.size(); i++) {  
        std::cout << i+1 << ". " << options[i] << '\n';  
    }  
}
```

No virtual here either — it already appeared once, in the header. Notice `ChoiceQuestion::print()` calls `Question::print()` first, then adds its own option list.

## Step 3: The Parameter Must Be a Pointer (or Reference) to the Superclass

```
void print(Rectangle* rectPtr) {  
    rectPtr->print();  
}
```

- This function takes a `Rectangle*` — a pointer to the *superclass*.
- Yet thanks to the virtual declaration, calling `rectPtr->print()` invokes whichever `print()` actually matches the object `rectPtr` points to — `Rectangle::print()` or `Square::print()` — decided *at run time*, based on the real object, not the pointer's declared type.

## Is-A Substitution: A Subclass Object Where a Superclass Is Expected

```
Rectangle rect(6, 8);
Square sq(7);
Square sq2 = sq;
sq2.setSide(6);

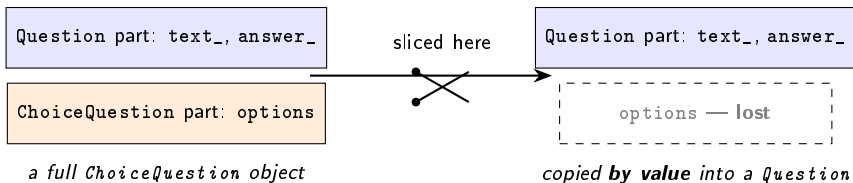
Rectangle* shapesPtr[3] = {&rect, &sq, &sq2};
for (int i = 0; i < 3; i++) {
    print(shapesPtr[i]);
}
```

- rect, sq, and sq2 have different actual types — but every Square *is a* Rectangle, so every one of their addresses fits in a Rectangle\* array.
- Same idea as: a student can be treated as a person, since a student *is a* person too.

- 1 Questions from Last Class
- 2 A Function vs. a Method
- 3 Review: The Is-A Relationship
- 4 Implementing Polymorphism in C++
- 5 The Slicing Pitfall**
- 6 Overload vs. Override

## The Catch: Object Slicing

- Is-a substitution also works if the superclass parameter is a **plain value**, not a pointer or reference — a Square can still be passed where a Rectangle is expected.
- **But:** the additional data members of the subclass are lost when a subclass object is *copied* into a variable or parameter of the superclass type. This is called **object slicing**.



```
void testQuestion(Question ques);           // by VALUE
void testQuestionByReference(const Question& ques); // by reference
void testQuestionByPointer(Question* quesPtr);  // by pointer
...
testQuestion(twoPlusThree);                 // COPY into a Question -> sliced
testQuestionByReference(twoPlusThree);      // no copy -> not sliced
testQuestionByPointer(&twoPlusThree);       // no copy -> not sliced
```

Only `testQuestion` copies its argument. That copy is constructed as a plain `Question` — the options vector that makes `twoPlusThree` a `ChoiceQuestion` simply has nowhere to go.

## Verified Output: Watch the Options Disappear

```
What is 2 + 3 ?  
Enter your answer: true
```

`testQuestion(twoPlusThree)` — passed **by value** — **no option list!**

```
What is 2 + 3 ?  
1. 3  
2. 4  
3. 5  
4. 6  
Enter your answer: true
```

`testQuestionByReference / testQuestionByPointer` — **no slicing** — options print correctly.

## Same Story with Rectangle/Square

```
Rectangle shapes[3] = {rect, sq, sq2};           // copied by value -> sliced
...
Rectangle* shapesPtr[3] = {&rect, &sq, &sq2};    // pointers -> not sliced
```

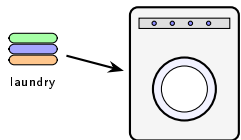
```
rectangle length: 6, rectangle width: 8    <- rect (fine, it IS a Rectangle)
rectangle length: 7, rectangle width: 7    <- sq, SLICED: prints as a
    rectangle!
rectangle length: 6, rectangle width: 6    <- sq2, SLICED too
...
square side: 7                             <- sq, via pointer: correct!
square side: 6                             <- sq2, via pointer: correct!
```

We ran this program and it behaved exactly as slicing predicts.

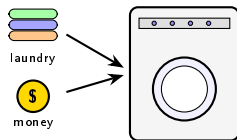
- 1 Questions from Last Class
- 2 A Function vs. a Method
- 3 Review: The Is-A Relationship
- 4 Implementing Polymorphism in C++
- 5 The Slicing Pitfall
- 6 Overload vs. Override

# Function/Method Overload: An Everyday Analogy

- Two or more functions/methods with the **same** name, **similar** functionality, but **different** parameters (in number, order, or type).
- The compiler distinguishes overloaded functions by their **parameter lists**, not by return type.



**home washer**  
`wash(laundry)`



**commercial washer**  
`wash(laundry, money)`

Same name, wash, but a **different number of parameters** — the compiler picks the matching version based on how many arguments you pass.

## Function/Method Overload: In C++

the washing machines, in code

```
void wash(int loads);           // home washing machine  
void wash(int loads, double money); // commercial washing machine
```

the same idea in your own code — three overloaded constructors, all named Rectangle

```
Rectangle();  
Rectangle(double length, double width);  
Rectangle(const Rectangle& rect);
```

Same name every time, but the compiler tells the different versions apart by their parameter lists.

# Method Override

- Happens in **inheritance**, and is what makes polymorphism useful.
- Means **re-defining** a method inherited from a superclass, inside a subclass.
- An overriding method must match the base virtual method's **name**, **parameters**, and **const** qualification, and normally has the **same return type**.

```
// Rectangle.hpp  
virtual void print() const;
```

```
// Square.hpp  
void print() const override;
```

Matching method signatures — override is optional but is good practice: it asks the compiler to double-check that this really does override a virtual method from the base class.

## An Override Doesn't Have to Call the Base Version

- `Square::print()` overrides `Rectangle::print()` — same name, same parameters (none), same `const`.
- But it does *not* call `Rectangle::print()` first. It replaces the behavior entirely, because a square doesn't have a separate length and width worth reporting.

```
// Rectangle.cpp
void Rectangle::print() const {
    std::cout << "rectangle length: "
        << length_ << ", rectangle
            width: "
        << width_ << '\n';
    ...
}
```

```
// Square.cpp
void Square::print() const {
    std::cout << "square side: "
        << side() << '\n';
    ...
}
```

Contrast this with `ChoiceQuestion::print()` (page 21), which deliberately calls `Question::print()` first, then adds its own output. Overriding lets you choose: *replace* the behavior, or *build on top of* it.

## Overload vs. Override, Side by Side

|                    | Overload                       | Override                  |
|--------------------|--------------------------------|---------------------------|
| Class relationship | none needed                    | requires inheritance      |
| Parameter list     | <b>must differ</b>             | <b>must match exactly</b> |
| Return type        | does not distinguish overloads | normally the same         |
| virtual needed?    | no                             | yes, in the base class    |
| Resolved           | at compile time                | at run time               |

Overload = several *different* tools with the same name. Override = a subclass provides its own version of an inherited virtual method.

- **Inheritance** is the is-a relationship: `ChoiceQuestion` is a `Question`; `Square` is a `Rectangle`.
- **Polymorphism**: write a function once, for the superclass, and it works with objects of different subclasses — as long as the parameter is a **pointer or reference** to the superclass, and the method is declared `virtual` in the superclass header (no need to repeat `virtual` in the `.cpp`).
- **Object slicing**: passing or copying a subclass object *by value* into a superclass-typed variable throws away everything the subclass added. Use pointers or references to avoid it.
- **Overload** = same name, different parameters, no inheritance required.  
**Override** = a subclass provides its own version of a virtual method inherited from the superclass.

Today's lecture slip: write `Square.cpp` yourself.

Everything you need is in these slides.